Design of a multi-threaded card game

We decided to split our code into 4 main classes: Card, CardDeck, Player, and CardGame. This division makes the most sense, as they are all the objects if the game were played in reality. CardGame is responsible for setting up the game, and starting threads. It also is the class that will be run by the user,

1 Classes

1.1 Card

The Card class is a simple class holding only one attribute. We decided to create it, instead of using integers in place of Card, to ensure the Card value remains non-negative. The Card class takes an integer value for its constructor and will throw an illegal argument exception if the card is negative.

1.2 CardDeck

Card Deck is a more complex class, it is responsible for implementing a circular queue, and for providing wait-notify logic for the Player threads. We implement this circular queue, with the variable top pointing to the start of the queue and tail pointing to the next empty space. We decided that a circular queue was best as it implements FIFO structure simply, and the max size of the deck is known.
CardDeck takes two values for the constructor, one for the number that represents the deck and one for the number of players in the game. We create a Card list with a size of $(4 \cdot n) + 1$, with n representing the number of players. The size of the list is chosen so that the deck cannot be full; as the players will always hold 4 cards, a single deck can only hold 4n cards at most. An alternative to this would be to prevent players from discarding to a full deck, pausing the Player threads so that they cannot discard, but this approach could cause issues with locking threads, potentially leading to a scenario where the game is completely paused due to deadlock. Having the size larger than necessary prevents any need for threads waiting for a full deck to become empty.
CardDeck does, however, wait threads when it is empty. Due to the non-deterministic nature of threads, one thread could easily empty a deck by moving its cards into another deck. If the queue is empty, then the CardDeck will call for the thread to wait, preventing the Player that draws from the CardDeck from trying to take a null value for Card. We note that the Players action remains atomic as they attempt to draw first, meaning that waiting here waits the Player before the action has truly begun.
The addCard() method will notify the waiting thread so that when a Card is added, the waiting Player can draw a Card and resume the thread.
We also notify threads in the notifyHasWon() method. This method is called by Player.run() by a winning Player, or by a Player when notified another has won. We call this method on the Player's discard deck, waking any threads waiting for a card to be added to that deck. Then, the drawCard() method will return null, failing the player's atomic action entirely. We decided to wake Players here as it ensures any waiting Player is notified in a simple manner. The printDeckToFile() method is only called once per deck at the end of the game. We decided to implement this in the Player class. We have each player call this method on their draw deck, ensuring that every deck prints their contents.

1.3 Player

Player is a class that inherits from the Thread class built into Java. We chose to inherit from Thread, as opposed to implementing Runnable since we did not need to implement any multi-inheritance.
The constructor for Player takes a number for the value of the Player, two CardDecks, one for drawing from and one for discarding to, an AtomicInteger that represents the number of the Player who wins the game, and a CyclicBarrier that prevents the thread from running until the game begins. The barrier stops the thread from starting until all threads have been created, preventing a Player from taking turns before all other threads have checked their initial hand. The Player constructor then creates a Card array of size 4, since the Player will only ever hold 4 cards due to the draw and discard needing to be one action. An integer value 'wanted' is also created to keep track of the number of cards Player has that it keeps hold of. (Cards that have a value equal to the number Player has been assigned). We decided to pass all of these argument to the constructor, as opposed

to making setters, as it makes the Player class definition cleaner.
The addCard method for Player takes in a Card value and adds it to any empty space in the Player hand. If the Card input is the same value as the Player number then the variable wanted is incremented. We use this for dealing to the player, as well as their action.
The private discard method will generate a random integer with a maximum value of the number of unwanted cards, (Cards that don't have the same value as the Player number) then go through the array and subtract from the value until 0 and return the Card at that position.This prevents the Player from discarding a wanted Card, while still randomly choosing a Card to discard. This could've been implemented by generating a random number between 0 and 3 and taking the Card at this index, then discarding it if the Card is unwanted or choosing a new random number. However, this approach could lead to the same number being generated several times before a Card is chosen, for example when there is only one unwanted Card, it is unlikely the random number will land on that Card.
Both addCard and discard are methods that are called by atomicDrawDiscard.
The purpose of this method is to make drawing and discard one atomic action, so that the number of Cards that Player holds is always exactly 4. When drawing a card, the deck the card is taken from is locked, so that other threads cannot discard into it. Similarly, when discarding, the discard deck is locked so that it is impossible for another thread to draw the Card a Player is currently discarding.
If the draw deck for a Player is empty, the Player releases the lock it has on the CardDeck and waits until notified by the CardDeck that another Player has discarded a Card into the deck, allowing the current Player to draw a Card.
If the action is interrupted by a Player winning, the action is either stopped before any changes are made, or the action is finished and then the thread is stopped. Alternatively, when interrupted the action could be reverted and the drawn card is put back into the CardDeck that it was drawn from. Neither solution has major downsides, but we went with the first as it introduced less complexity.
This method is thread safe as two Players will never be able to see the same Card at the same time or overwrite a Card. Other threads may be able to take actions such as drawing from a CardDeck that a second Player is about to discard to, and has already drawn from it's respective draw deck, but the first Player will never see the Card that the second Player is discarding, as the CardDeck would be locked while that action takes place. Hence there will never be two pointers to the same Card object.
After the method has finished drawing and discarding, the Player will write the action to a text file, appending to the current one.

1.4 CardGame

The CardGame class is responsible for getting user input, setting up the game, and starting the threads. It takes user input in the main() method. The constructor takes only the number of required players for the game. It then creates an AtomicInteger, used as a flag to notify players when another has won, and a CyclicBarrier descibed in 1.3. It also creates new Players and CardDecks. This basic setup is required for every game, so we decided to put it in the constructor, forcing it to be called when CardGame is created.
We decided to check a pack's validity at the same time as it is read, in the readPack() method. If it is valid, it returns an array of cards, otherwise it throws an exception. Throwing an exception allows us to handle output messages to the user in the main() method.
We seperated the deal() method from the constructor, as the pack has not been read in, and from readPack(). This ensures that the code is cleaner, as readPack() is likely to be called many times if the user gives an invalid pack. The cards are dealt in the round robin fashion specified.
Every player thread is started in the runGame() method, as opposed to main(). This exposes the running of the game to make system tests easier. We call CyclicBarrier.await() on the main thread. This starts every thread at the same time. We want this functionality to allow every player to check if their initial hand as mentioned in 1.3. We use join() on player 1, as this is a player that will be present in every gamee. This stops the main thread from continuing execution until player 1 has finished, we want this to occur so we can output the winning player from main().
Finally, the main() method is responsible for getting user input. We check that each user input is valid, and then read in a pack. We handle the errors thrown by readPack() by outputting a message to the user, and asking for a new valid pack. We then run the game.

2 Testing

We designed our test suite using JUnit 4.13.2. We opted for JUnit4 over 5 or 6, as this version is focused on by the module. Further, we decided the extra features from later versions would not be needed to test the card

game.

2.1 Unit tests

We test all methods, barring simple getters, in the public interface for correct behaviour in edge cases, and required errors. As is convention with JUnit4, we have a test class for each class. We also use @AfterClass to delete any output files created by the tests, ensuring a clean environment after tests have been run.
Card, being a simple class, does not have many tests. The main important test checks if the constructor throws an error when passed a negative integer.
CardDeck calls for more unit tests. Primarily, we test that the correct number of cards are added, and that the FIFO strucuture of the deck works correctly. We also test that the output file is formatted correctly, and contains the correct cards. We decided that the wait-notify thread functionality with empty decks, would be easier to test if we considered it an integration test, rather than a unit test. As such, we decided to test for this in TestPlayer. There are few edge cases to test for here, as the majority of CardDeck is simply a circular queue implementation.
The TestPlayer class contains both unit tests for the Player class, and some integration tests between Player and CardDeck. It also uses an @AfterClass method to remove the two files created while the tests are run. We unit test both that the draw-discard action is atomic, i.e. that after the action, the player has 4 cards, and one of them has been replaced by a new card. We test that the player can win with 4 of any cards, this is important functionality.
Finally, TestCardGame has many unit tests for the setup of the game, and several system tests. The majority of the tests are around the validity of a pack, as this has important conditions that should be adhered to. We designed tests that expect an exception to be thrown by the readPack() method, which throws a seperate exception for each potential invald pack condition. This allows us to test several packs, in the TestPacks folder, that are invalid by simply calling CardGame.readPack(). We also check that the constructor correctly sets up the AtomicInteger winner flag.

2.2 Integration tests

As mentioned before, the test player class contains some integration tests. We test that Player is correctly paused during a draw-discard action if the draw deck is empty. To do this, we create two anonymous classes from Thread. This allows us to run two players at once, while forcing them both to call atomicDrawDiscard only once. This ensures that the tests can be run on a non-deterministic method, as otherwise the entire game would have to be run. Here, we know only one card in each player hand should be changed. This test ensures the wait-notify functionality works.

2.3 System tests

TestCardGame also tests the system as a whole, on several test packs. These test packs are designed so that only a player, or a range of players, have enough preferred cards to win. We decided to design them this way to allow us to assert some player, or range of players has won, ensuring correct behaviour in the system.
We test for edge cases, such as a player immediately winning and games where only 1 player can win. We also test normal games, with a small number of players and a large number of players. These games have a range of players who cannot win, allowing the assert to be used.
Due to the non-deterministic nature of this game, a test could succeed one time, and fail another time. To counteract this, we decided that in each test, the game should be run number of times, making the probability of differing test results very low. We decided that games with a small number of players should be run 100 times, and a with large number of players 1000 times. This makes the probability of differing results acceptably low.

2.4 Limitations of tests

The main limitation of the tests is due to the non-determinism of the game. Since a singular output cannot be predicted given a pack, certain system tests are impossible to implement. We made allowences for this as detailed above, however, some tests still cannot be implemented.
Another limitation is due to the requirement of the draw-discard action to be atomic. Because of this, we made the decision not to expose the private method discard() to avoid unwanted calls to it. This does, however, mean that testing the discard functionality, also requires testing the drawing functionality by calling atomicDrawDiscard().